

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
ТЕМА: Ахо-Корасик

Студент гр. 4343

Стрижков И.А.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Цель работы.

Изучить принцип работы алгоритма Ахо-Корасик. Написать две программы, решающие задачи поиска набора образцов в тексте и одного образца с джокером.

Задание №1.

Разработайте программу, решающую задачу точного поиска набора образцов.

Входные данные:

Первая строка содержит текст ($T, 1 \leq |T| \leq 100000$).

Вторая - число n ($1 \leq n \leq 3000$), каждая следующая из n строк содержит шаблон из набора $P = \{p_1, \dots, p_n\}$ $1 \leq |p_i| \leq 75$

Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Выходные данные:

Все вхождения образцов из P в T .

Каждое вхождение образца в текст представить в виде двух чисел - i p

Где i - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p (нумерация образцов начинается с 1).

Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

Sample Input:

NTAG

3

TAGT

TAG

T

Sample Output:

2 2

2 3

Задание №2.

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с *джокером*.

В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблон образцу P необходимо найти все вхождения P в текст T .

Например, образец $ab??c?$ с джокером $?$ встречается дважды в тексте $xabvccbababcax$.

Символ джокер не входит в алфавит, символы которого используются в T . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида $???$ недопустимы.

Все строки содержат символы из алфавита $\{A,C,G,T,N\}$

Входные данные:

Текст ($T, 1 \leq |T| \leq 100000$)

Шаблон ($P, 1 \leq |P| \leq 40$)

Символ джокера

Выходные данные:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер).

Номера должны выводиться в порядке возрастания.

Sample Input:

ACTANCA

A\$\$\$A\$

\$

Sample Output:

1

Вариант 1: На месте джокера может быть любой символ, за исключением заданного.

Описание алгоритма Ахо-Корасик

Алгоритм основан на построении префиксного дерева (бора) из заданных паттернов с последующим обходом переданного текста. На входе известны только сами паттерны.

На первом этапе строится бор из символов паттернов. После этого в него добавляются суффиксные и терминальные ссылки. Суффиксная ссылка каждой вершины указывает на вершину, соответствующую наидлиннейшему собственному суффиксу текущей строки, который присутствует в боре. Корень бора в качестве исключения ссылается на себя. Суффиксные ссылки необходимы для случаев, когда при обработке очередного символа текста переход по боровым рёбрам невозможен - либо паттерн найден полностью, либо нужного ребра просто нет. Помимо суффиксных, существуют терминальные ссылки, которые указывают на ближайшую терминальную вершину, достижимую по цепочке суффиксных ссылок.

После построения автомата выполняется поиск в тексте. Каждый символ текста вызывает переход из текущего состояния в новое - либо по ребру бора, либо по суффиксной ссылке. Благодаря этому автомат никогда не «застревает»: даже достигнув конца паттерна, следующий переход по

суффиксной ссылке вернёт в корень или в более раннюю вершину. Терминальные ссылки позволяют эффективно обнаруживать все вхождения, не пропуская паттерны, которые являются суффиксами других паттернов. В итоге алгоритм находит все вхождения всех паттернов в тексте.

Сложность по времени:

Временная сложность алгоритма составляет $O(\sum |p_i| + |T|)$, где $\sum |p_i|$ - суммарная длина всех паттернов, а $|T|$ - длина текста. Построение бора занимает $O(\sum |p_i|)$, поскольку каждый символ каждого паттерна обрабатывается ровно один раз. Обход текста для поиска вхождений выполняется за $O(|T|)$, так как из каждого состояния автомата переход происходит за константное время благодаря заранее построенным суффиксным и терминальным ссылкам.

Сложность по памяти:

Сложность алгоритма по памяти - $O(\sum |p_i| * k)$, где k - размер алфавита, поскольку каждая вершина хранит множество своих детей, и в худшем случае его дети – все символы данного алфавита.

Описание модифицированного алгоритма

Построение бора и суффиксных ссылок выполняется так же, как в основном алгоритме, однако в качестве паттернов используются подстроки шаблона, разделённые символом джокера. Логика поиска этих подстрок в тексте практически не отличается от базового случая: после нахождения всех совпадений получаем список позиций и подстрок, потенциально являющихся частями искомого шаблона, и для каждого такого совпадения вычисляется позиция, с которой должен начинаться весь шаблон, чтобы данная подстрока оказалась на своём месте. Все такие позиции добавляются в множество кандидатов, которое автоматически устраняет дубликаты и не требует массива длиной $|T|$, в отличие от подхода со счётчиками. После сбора кандидатов

выполняется финальная посимвольная проверка всего шаблона с текстом начиная с каждой позиции-кандидата, при этом учитывается особенность варианта 1: если символ шаблона является джокером, соответствующий символ текста не должен быть равен заданному запрещённому символу, иначе символы должны совпадать; успешно прошедшие проверку позиции добавляются в итоговый список результатов.

Сложность по времени для модифицированного алгоритма:

Сложность алгоритма по времени - $O(\sum |T| + |P| + K + |P|)$ совпадает с оригинальным алгоритмом.

Сложность по памяти для модифицированного алгоритма:

Объём используемой памяти составляет $O(\sum |p_i| * k + K)$, где k — размер алфавита, а K — количество найденных кандидатов. По сравнению с оригинальным алгоритмом добавляется множество кандидатов, размер которого не превышает K . В отличие от подхода со счётчиками по всем позициям текста, требующего массива длины $|T|$, здесь память пропорциональна количеству найденных кандидатов, что на практике существенно меньше длины текста.

Описание функций

1. `getCharIndex` — преобразует символ алфавита в его числовой индекс от 0 до 4, возвращая -1 для символов, не входящих в алфавит.
2. `getCharFromIndex` — выполняет обратное преобразование: по числовому индексу возвращает соответствующий символ алфавита, используется при выводе промежуточных данных.
3. `Node` — структура, описывающая одну вершину бора: хранит массив переходов по символам алфавита, суффиксную ссылку, массив переходов автомата и список номеров частей шаблона, заканчивающихся в данной вершине.

4. `addPart` — добавляет одну безджокерную часть шаблона в бор, создавая при необходимости новые вершины и помечая конечную вершину как терминальную с сохранением стартовой позиции и длины части.
5. `buildLinks` — строит суффиксные ссылки и переходы автомата для всех вершин бора с помощью обхода в ширину (BFS).
6. `printTrie` — выводит структуру построенного бора с информацией о каждой вершине: исходящие рёбра и список заканчивающихся в ней частей шаблона.
7. `printAutomaton` — выводит построенный автомат: для каждой вершины печатает значение суффиксной ссылки и все переходы автомата по символам алфавита.

Тестирование.

Таблица 1 – Тестирование алгоритма Ахо-Корасик

Входные данные	Выходные данные
NTAG 3 TAGT TAG T	2 2 2 3
GCATGCAT 3 GCAT CAT AT	1 1 2 2 3 3 5 1 6 2 7 3
AACTGAAC 2 AAC TG	1 1 4 2 6 1

Таблица 2 – Тестирование алгоритма поиска с джокером

Входные данные	Выходные данные
ACTANCA A\$\$\$ \$	вхождений нет

T	
AACTGAACT	1
A\$CT	6
\$	
G	
GTATNGCAT	1
G\$AT	
\$	
C	

```

ACTANCA
A$$A$
$
T
Введите текст T: Введите шаблон P: Введите символ джокера: Введите запрещенный символ для джокера:
=== РАЗБИЕНИЕ ШАБЛОНА ===
Часть 0: "A" (начало в шаблоне: 0, длина: 1)
Часть 1: "A" (начало в шаблоне: 3, длина: 1)

=== СТРУКТУРА БОРА ===
Вершина 0: A->1
Вершина 1: нет исходящих ребер [ТЕРМИНАЛ: "A" "A" ]

=== АВТОМАТ (СУФИКСНЫЕ ССЫЛКИ И ПЕРЕХОДЫ) ===
Вершина 0: link=0 | Переходы: A->1 C->0 G->0 T->0 N->0
Вершина 1: link=0 | Переходы: A->1 C->0 G->0 T->0 N->0

=== ПРОЦЕСС ПОИСКА В ТЕКСТЕ ===
Шаг 1: символ 'A', переход 0 -> 1
  Найдена часть "A" (конец на позиции 1), возможное начало шаблона: 1
  Найдена часть "A" (конец на позиции 1), но начало шаблона выходит за границы (-2) -> отброшено
Шаг 2: символ 'C', переход 1 -> 0
Шаг 3: символ 'T', переход 0 -> 0
Шаг 4: символ 'A', переход 0 -> 1
  Найдена часть "A" (конец на позиции 4), возможное начало шаблона: 4
  Найдена часть "A" (конец на позиции 4), возможное начало шаблона: 1
Шаг 5: символ 'N', переход 1 -> 0
Шаг 6: символ 'C', переход 0 -> 0
Шаг 7: символ 'A', переход 0 -> 1
  Найдена часть "A" (конец на позиции 7), возможное начало шаблона: 7
  Найдена часть "A" (конец на позиции 7), возможное начало шаблона: 4

=== ПРОВЕРКА КАНДИДАТОВ ===
Проверка кандидата pos=1:
  Позиция 0: 'A' == 'A' -> ОК
  Позиция 1: джокер на 'C' (не запрещен) -> ОК
  Позиция 2: джокер совпал с запрещенным символом 'T' -> не подходит
Кандидат pos=4 отброшен: выход за границы текста
Кандидат pos=7 отброшен: выход за границы текста

=== РЕЗУЛЬТАТЫ ПОИСКА ===
Вхождений не найдено.

```

Рисунок 1 – Пример вывода поиска модифицированного алгоритма

Исследование.

Для проведения исследования, оценим рост временной сложности создания бора и поиска в тексте и сравним с теоретическими данными.

Таблица 3 – Исследование эффективности алгоритма по времени

Суммарный размер паттернов	Время создания бора
10	0.000008
100	0.000086
1000	0.000904
10000	0.009003
100000	0.094245
Размер текста	Время поиска вхождений
100	0.000024
1000	0.000210
10000	0.001938
100000	0.018878
1000000	0.190350

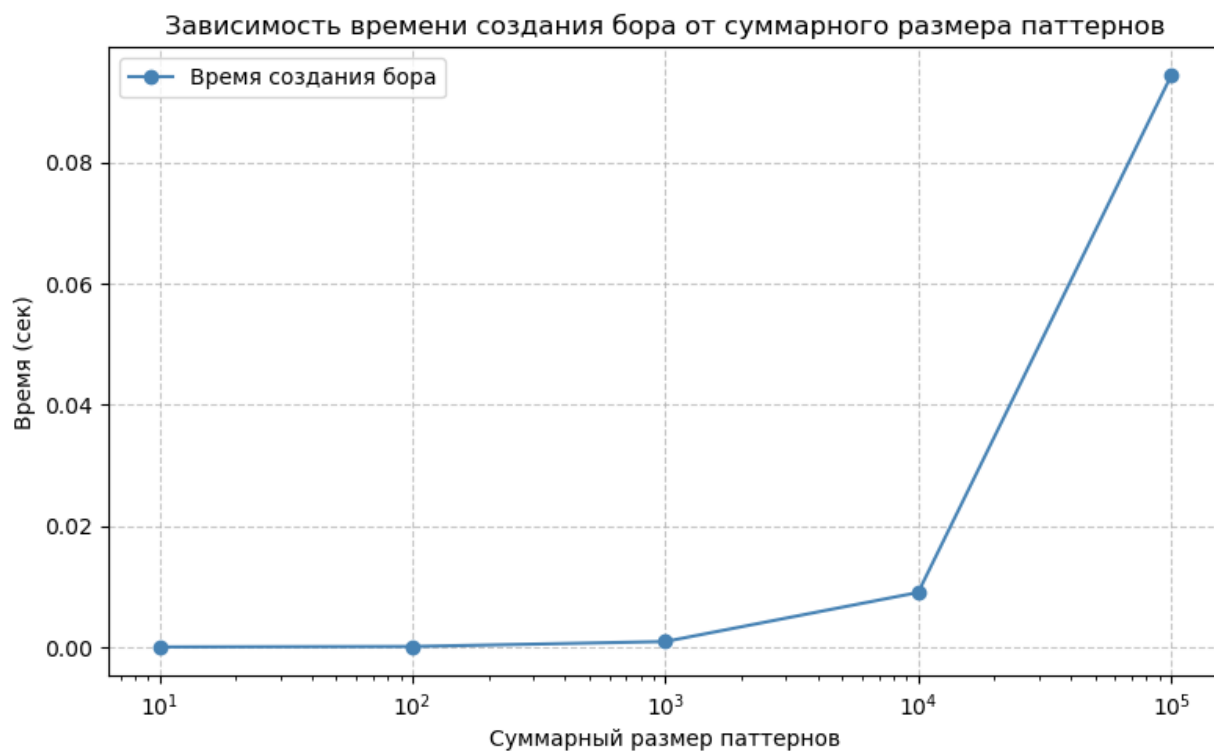


Рисунок 6 – Зависимость времени создания бора от суммарного размера паттернов



Рисунок 7 – Зависимость времени поиска от размера текста

Можно сделать вывод, что время построения бора и поиска в тексте линейно и зависит соответственно от размеров паттернов и текста, что подтверждает теоретическую оценку. Это можно увидеть в таблице или на графике.

Выводы.

Изучен принцип работы алгоритма Ахо-Корасик. Написана программа, решающая задачу точного поиска одного образа с джокером в тексте. Шаблон разбивается на подстроки, разделённые символом джокера, которые добавляются в бор, после чего строятся суффиксные ссылки и переходы автомата. Поиск частей шаблона в тексте выполняется с помощью построенного автомата, а для каждой найденной части вычисляется возможная позиция начала всего шаблона и проводится финальная проверка с учётом того, что джокер не может совпасть с заданным запрещённым символом.